

Persistent Conversation History

1. What Is Persistent State?

Persistent state means application data is stored outside temporary application memory so that it can be recovered later.

Simple idea:

Python Program



Stores Data



Program Stops



Program Starts Again



Load Previous Data

So:

Persistent State



Data survives beyond
the current program execution

2. In-Memory vs Persistent History

In-Memory History

Python Program



History in RAM



Program Stops



History is Lost

Example:

```
store = {}
```

or:

```
history = []
```

These are normally available only while the current program is running.

Persistent History

Python Program



Save History Outside Program Memory



Program Stops



Program Starts Again



Read Saved History



Continue Conversation

3. Possible Persistent Stores

Conversation history can be stored using:

1. Files
2. SQLite
3. PostgreSQL
4. Cloud databases
5. Other persistent storage systems

Simple idea:

Conversation History



Persistent Storage



Can Be Recovered Later

4. Why Do We Need Serialization?

Our conversation history contains LangChain message objects such as:

HumanMessage(...)

and:

AIMessage(...)

But when we want to save these objects into a file or database, we need a suitable storable format.

Therefore, we use:

Serialization

5. LangChain Message Serialization and Deserialization

LangChain provides useful functions such as:

`message_to_dict()`

and:

`messages_from_dict()`

These help us convert between:

LangChain Message Objects



Python Dictionaries

6. What Is Serialization?

Serialization means converting an object into a format that can be easily stored or transferred.

Simple flow:

LangChain Message Object



Serialization



Python Dictionary



JSON



Can Be Stored

Simple definition:

Serialization = Object → Storable Format

7. First Serialization Example

```
from langchain_core.messages import (  
    HumanMessage,  
    message_to_dict  
)
```

```
message = HumanMessage(  
    content="Hello Durga Sir"  
)
```

```
data = message_to_dict(  
    message
```

)

print(data)

Possible output:

```
{  
  'type': 'human',  
  
  'data': {  
    'content': 'Hello Durga Sir',  
    'additional_kwargs': {},  
    'response_metadata': {},  
    'type': 'human',  
    'name': None,  
    'id': None  
  }  
}
```

8. What Does `message_to_dict()` Do?

`message_to_dict()`

converts one LangChain message into a Python dictionary.

Concept:

HumanMessage



`message_to_dict()`



Python Dictionary

Example:

Message Object



Serializable Dictionary

9. What Does "Serializable" Mean?

Serializable means the data is in a form that can be easily saved or transferred.

Example:

HumanMessage Object



Python Dictionary



JSON



File / Database

A Python or LangChain object cannot always be directly stored in a JSON file.

Therefore, we first convert it into a simpler data representation.

10. Easy Serialization Formula

OBJECT



SERIALIZATION



STORABLE FORMAT

For our LangChain example:

LangChain Message



Dictionary



JSON

11. What Is Deserialization?

Deserialization means converting stored data back into the original type of object.

Simple flow:

JSON / Dictionary



Deserialization



LangChain Message Object

Simple definition:

Deserialization = Stored Format → Object

12. LangChain Deserialization

LangChain provides:

`messages_from_dict()`

It converts message dictionaries back into LangChain message objects.

Concept:

Message Dictionaries



messages_from_dict()



LangChain Messages

13. Serialization vs Deserialization

Serialization

Object



Dictionary



JSON

Deserialization

JSON



Dictionary



Object

So:

Serialization

Object → Stored Format

and:

Deserialization

Stored Format → Object

14. Complete Conversation Persistence Flow

HumanMessage / AIMessage



message_to_dict()



Python Dictionary



json.dumps()



JSON String



Save Data



Load Data



`json.loads()`



Python Dictionary



`messages_from_dict()`



HumanMessage / AIMessage

15. Complete Program — Message to JSON and Back to Message

```
import json
```

```
from langchain_core.messages import (
```

```
    HumanMessage,
```

```
    AIMessage,
```

```
    message_to_dict,
```

```
    messages_from_dict
```

```
)
```

```
# =====
```

```
# 1. CREATE LANGCHAIN MESSAGES
```

```
# =====
```

```
messages = [  
    HumanMessage(  
        content="My name is Durga."  
    ),  
  
    AIMessage(  
        content="Nice to meet you, Durga."  
    )  
]
```

```
print("ORIGINAL LANGCHAIN MESSAGES")
```

```
print("-----")
```

```
for message in messages:
```

```
print(  
    type(message).__name__,  
    ":",  
    message.content  
)
```

```
# =====  
# 2. MESSAGES -> PYTHON DICTIONARIES  
# =====
```

```
serialized = [  
    message_to_dict(message)  
    for message in messages  
)
```

```
print("\nPYTHON DICTIONARIES")  
print("-----")
```

```
print(serialized)
```

```
# =====
```

```
# 3. PYTHON DICTIONARIES -> JSON STRING
```

```
# =====
```

```
json_string = json.dumps(  
    serialized,  
    indent=2  
)
```

```
print("\nJSON STRING")
```

```
print("-----")
```

```
print(json_string)
```

```
# =====
```

4. JSON STRING -> PYTHON DICTIONARIES

=====

```
python_data = json.loads(  
    json_string  
)
```

```
print("\nBACK TO PYTHON DICTIONARIES")
```

```
print("-----")
```

```
print(python_data)
```

=====

5. PYTHON DICTIONARIES

-> LANGCHAIN MESSAGES

=====

```
restored_messages = messages_from_dict(
```

```

python_data
)

print("\nRESTORED LANGCHAIN MESSAGES")
print("-----")

for message in restored_messages:

    print(
        type(message).__name__,
        ":",
        message.content
    )

```

16. Step 1 — Create LangChain Messages

We first create:

```

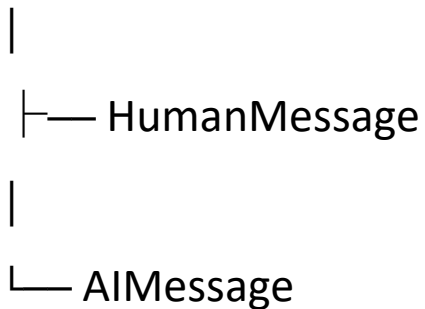
messages = [
    HumanMessage(
        content="My name is Durga."
    )
]

```

```
),  
  
    AIMessage(  
        content="Nice to meet you, Durga."  
    )  
]
```

Conceptually:

messages



These are LangChain message objects.

17. Step 2 — Message Objects to Python Dictionaries

We use:

```
serialized = [  
    message_to_dict(message)  
    for message in messages  
]
```

Concept:

HumanMessage

AIMessage



message_to_dict()



Python Dictionaries

18. Why Use a List Comprehension?

We have multiple messages.

Therefore:

message_to_dict(message)

must be applied to every message.

So we use:

```
[  
    message_to_dict(message)  
    for message in messages  
]
```

Concept:

Message 1 → Dictionary 1

Message 2 → Dictionary 2

Message 3 → Dictionary 3

19. Step 3 — Python Dictionaries to JSON String

We use:

```
json_string = json.dumps(  
    serialized,  
    indent=2  
)
```

The important method is:

```
json.dumps()
```

It converts:

Python Object



JSON String

20. What Does json.dumps() Mean?

```
json.dumps()
```

means:

Python Object



JSON-formatted String

Example:

```
data = {  
    "name": "Durga"  
}
```

After:

```
json.dumps(data)
```

we get JSON text representing that Python data.

21. What Does indent=2 Mean?

indent=2

means:

Use two spaces for each indentation level while formatting the JSON output.

Without indentation, JSON may look compact.

With:

indent=2

it becomes easier for humans to read.

22. Step 4 — JSON String Back to Python Data

We use:

```
python_data = json.loads(  
    json_string  
)
```

Here:

`json.loads()`

converts:

JSON String



Python Object

In our example, it gives us Python dictionaries/lists again.

23. What Does `json.loads()` Mean?

`json.loads()`

means:

JSON String



Python Data

Easy memory:

loads

JSON → Python

24. Step 5 — Dictionaries Back to LangChain Messages

Now we have Python dictionaries again.

We use:

```
restored_messages = messages_from_dict(  
    python_data  
)
```

Concept:

Python Message Dictionaries



messages_from_dict()



HumanMessage / AIMessage

25. Restored Messages

We can print them:

for message in restored_messages:

```
print(
    type(message).__name__,
    ":",
    message.content
)
```

Possible output:

HumanMessage : My name is Durga.

AIMessage : Nice to meet you, Durga.

So we successfully came back to LangChain message objects.

26. Complete Transformation

Start:

HumanMessage

AIMessage

Then:

Dictionary

Then:

JSON String

Then back to:

Dictionary

Finally:

HumanMessage

AIMessage

27. Master Flow

messages



message_to_dict()



Python Dictionaries



json.dumps()



JSON String



`json.loads()`



Python Dictionaries



`messages_from_dict()`



LangChain Messages

28. Four Important Functions

1. `message_to_dict()`

LangChain Message



Python Dictionary

Example:

`message_to_dict(message)`

2. `json.dumps()`

Python Object



JSON String

Example:

```
json.dumps(data)
```

3. json.loads()

JSON String



Python Object

Example:

```
json.loads(json_string)
```

4. messages_from_dict()

Message Dictionaries



LangChain Messages

Example:

```
messages_from_dict(data)
```

29. Easy Comparison

Function	Input	Output
message_to_dict()	LangChain Message	Python Dictionary
json.dumps()	Python Object	JSON String
json.loads()	JSON String	Python Object
messages_from_dict()	Message Dictionaries	LangChain Messages

30. Student Memory Trick

Remember:

dumps



Python → JSON String

loads



JSON String → Python

Very simple:

dumps = TO JSON

loads = FROM JSON

31. Serialization Side

LangChain Message



message_to_dict()



Dictionary



json.dumps()



JSON String

So:

Serialization Side

Message



Dictionary



JSON String

32. Deserialization Side

JSON String



`json.loads()`



Dictionary



`messages_from_dict()`



LangChain Message

So:

Deserialization Side

JSON String



Dictionary



Message

33. Why Is This Important for Persistent History?

Suppose our conversation contains:

HumanMessage

AIMessage

HumanMessage

AIMessage

These messages exist as application objects.

To store them persistently:

LangChain Messages



Serialize



Store in File / Database

Later:

Read Stored Data



Deserialize



LangChain Messages



Continue Conversation

34. Persistent Conversation Architecture

Conversation



LangChain Messages



Serialize



Persistent Storage



Program Stops



Program Starts Again



Load Stored Data



Deserialize



LangChain Messages



Continue Conversation

35. Connection with Previous Sessions

Previously, we used:

InMemoryChatMessageHistory

This stores messages in application memory.

Now we are moving towards:

Persistent Conversation History

The main difference is:

In-Memory



Temporary

versus:

Persistent



Can Be Recovered Later

36. In-Memory vs Persistent

In-Memory History

Stored in current application memory

Easy to use

May disappear when process stops

Good for demos/testing

Persistent History

Stored outside temporary application memory

Requires storage mechanism

Can be loaded again later

Useful for longer-lived applications

37. Important Point — Dictionary vs JSON String

Do not confuse these two.

Python dictionary:

```
{  
    "name": "Durga"  
}
```

JSON string is text representation.

Concept:

Python Dictionary



`json.dumps()`



JSON String

And:

JSON String



`json.loads()`



Python Dictionary

38. Important Point — `message_to_dict()` Does Not Directly Produce a File

`message_to_dict(message)`

produces a Python dictionary.

It does not automatically save anything to disk.

Concept:

Message



`message_to_dict()`



Dictionary

To actually persist the data, we still need to save that data using a file, database, or another storage mechanism.

39. Important Point — Serialization Is Not Persistence by Itself

Very important:

Serialization

≠

Persistence

Serialization prepares data for storage.

Persistence means actually storing it somewhere that survives beyond the current process.

So:

Object



Serialize



Storable Data



Save to File / Database



Persistent Data

40. Important Key Points

1. Persistent state is stored outside temporary application memory.
2. Persistent conversation history can be recovered after the program restarts.
3. Files and databases can be used as persistent stores.
4. Serialization converts an object into a storable or transferable format.
5. Deserialization converts stored data back into an object.
6. `message_to_dict()` converts one LangChain message into a Python dictionary.
7. `messages_from_dict()` converts message dictionaries back into LangChain message objects.
8. `json.dumps()` converts Python data into a JSON string.
9. `json.loads()` converts a JSON string back into Python data.
10. `indent=2` makes JSON output easier to read.
11. Serialization does not automatically mean the data has been permanently stored.

12. Persistent storage requires actually writing the serialized data to a file, database, or another persistent store.

41. Common Mistake — Thinking Dictionary Is Already JSON Text

Wrong.

```
{  
    "name": "Durga"  
}
```

is a Python dictionary.

After:

```
json.dumps(...)
```

we get a JSON string.

42. Common Mistake — Confusing dumps and loads

Remember:

dumps

Python → JSON

loads

JSON → Python

43. Common Mistake — Thinking message_to_dict() Stores the Message Permanently

Wrong.

message_to_dict()

only converts the message into a dictionary.

You still need a persistent storage mechanism.

44. Common Mistake — Thinking Serialization and Deserialization Are the Same

They are opposite operations.

Serialization

Object → Stored Format

Deserialization

Stored Format → Object

45. Final Master Architecture

LANGCHAIN MESSAGE

|

↓

message_to_dict()

|



Python Dictionary



`json.dumps()`



JSON String



Persistent Storage



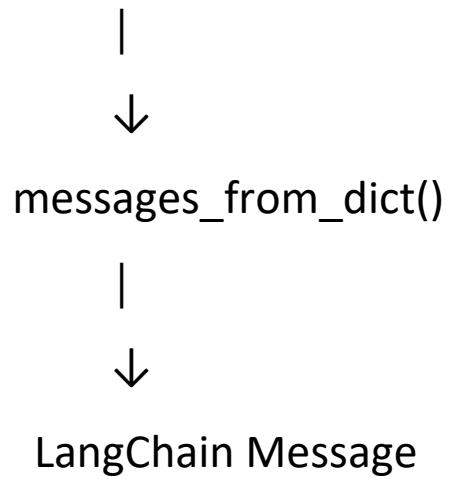
Load Data



`json.loads()`



Python Dictionary



46. Final Memory Formula

SERIALIZATION

=====

Message



Dictionary



JSON

DESERIALIZATION

=====

JSON



Dictionary



Message

47. Student Memory Trick

`message_to_dict()`

Message → Dictionary

`json.dumps()`

Python → JSON String

`json.loads()`

JSON String → Python

`messages_from_dict()`

Dictionary → Messages

48. One-Line Conclusion

Persistent conversation history requires storing conversation data outside temporary application memory, and serialization helps convert LangChain message objects into storable data while deserialization converts the stored data back into LangChain messages.

